Lec-24

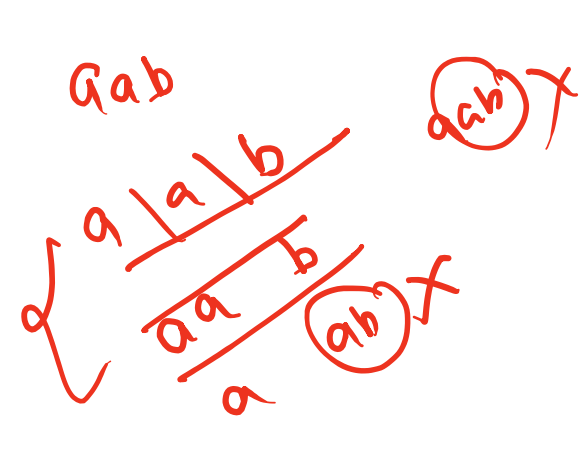
03/07/26 7:11 PM

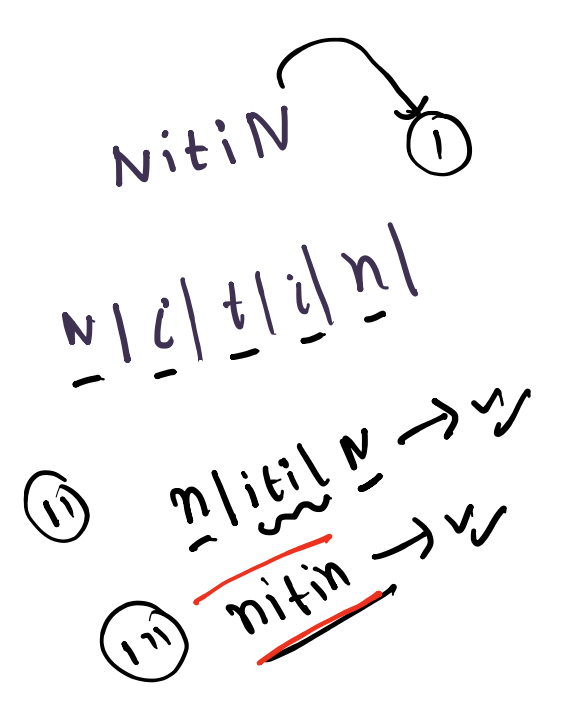
Given a string `s`, partition `s` such that every substring of the partition is a **palindrome**. Return *all possible palindrome partitioning of* `s`.

s aab

Example 1:

```
Input: s = "aab"
Output: [["a","a","b"],["aa","b"]]
```

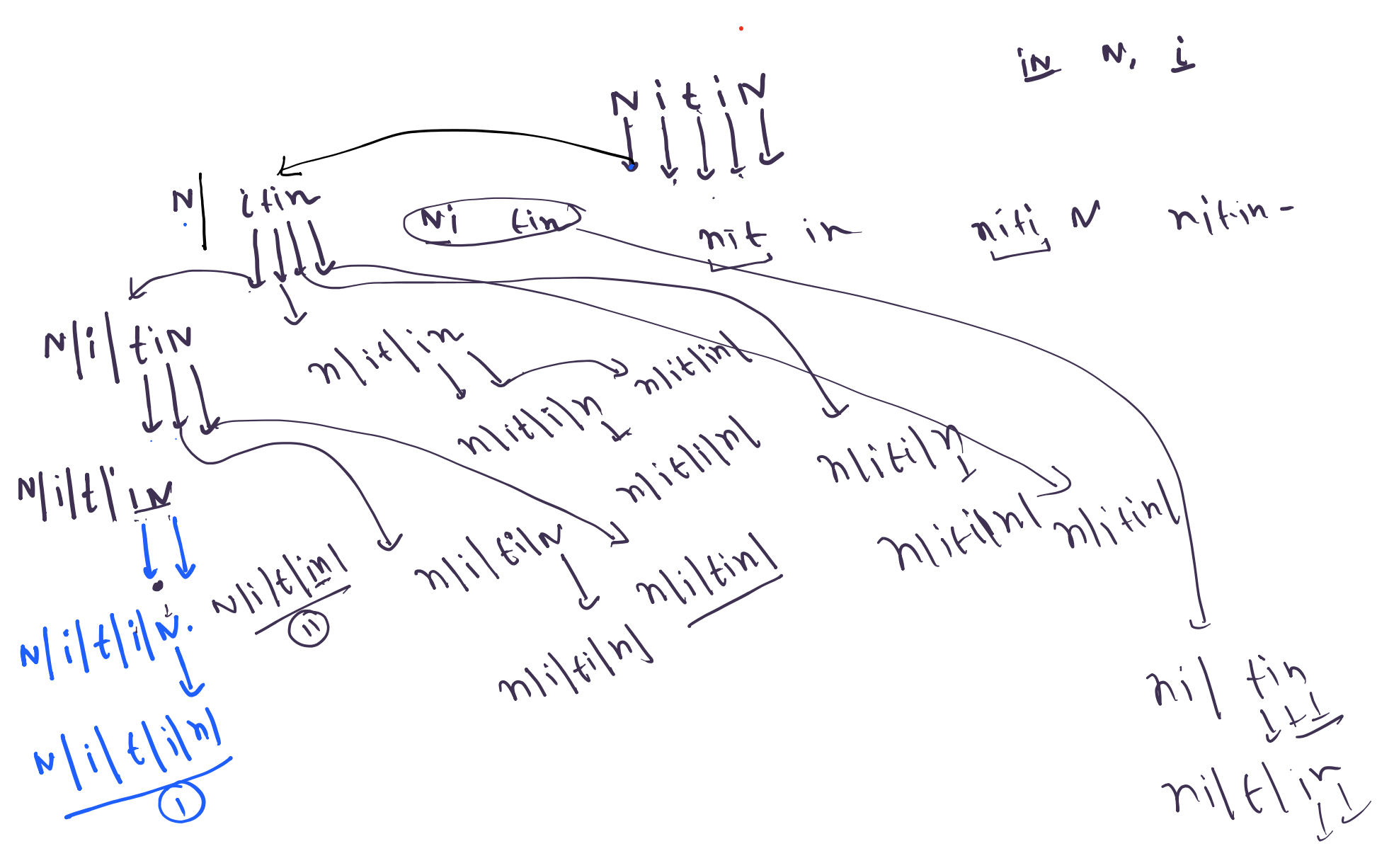

aab
a|a|b
aa b
a
ab

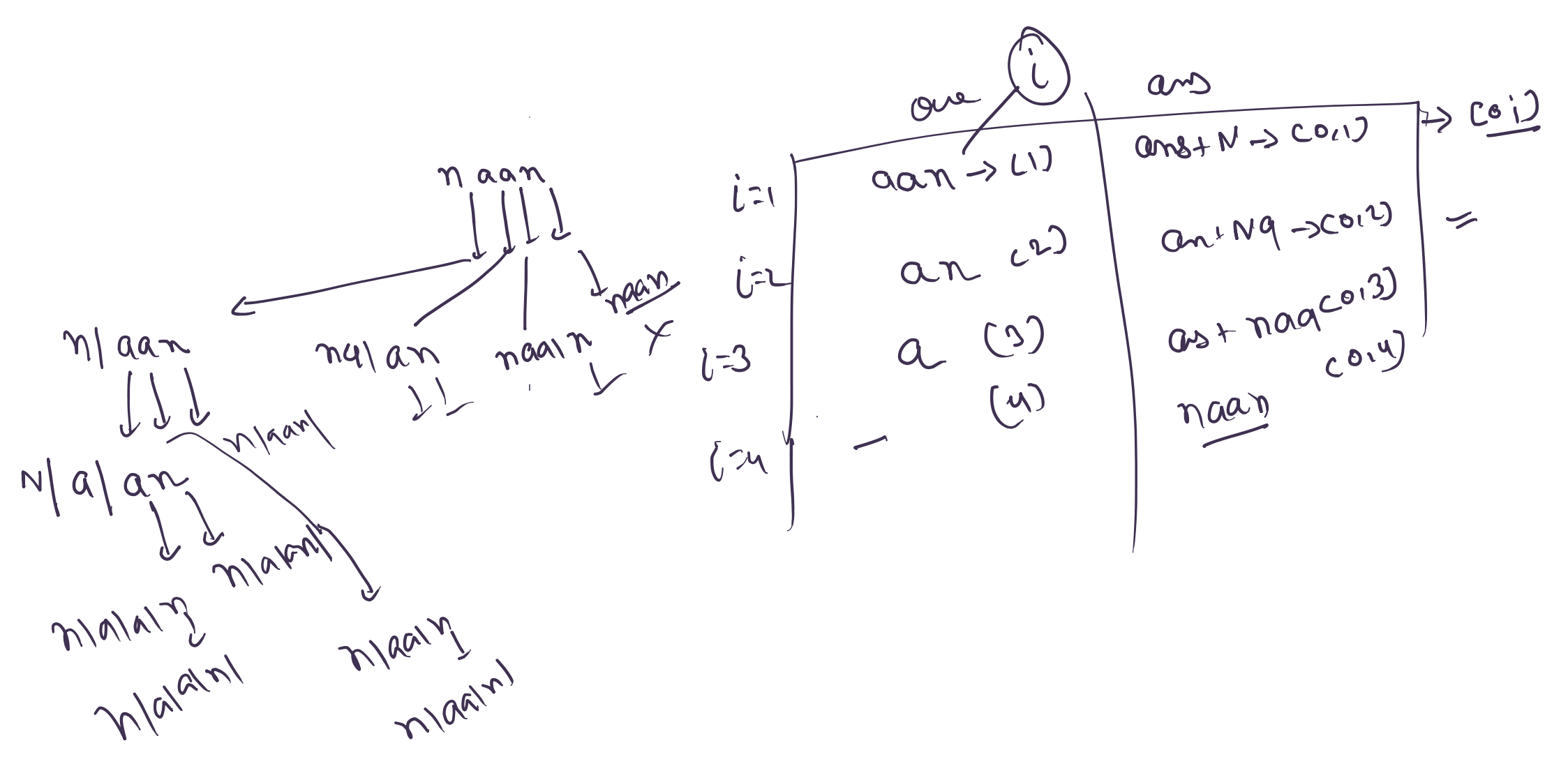

NitiN
1
N|i|t|i|n|
n|iti|n
nitin

```
n|i|t|i|n|
n|i|t|in|
n|i|ti|n|
n|i|tin|
n|it|i|n|
n|it|in|
n|iti|n|
n|itin|
ni|t|i|n|
ni|t|in|
ni|ti|n|
ni|tin|
nit|i|n|
nit|in|
niti|n|
nitin|
```

NLtIN

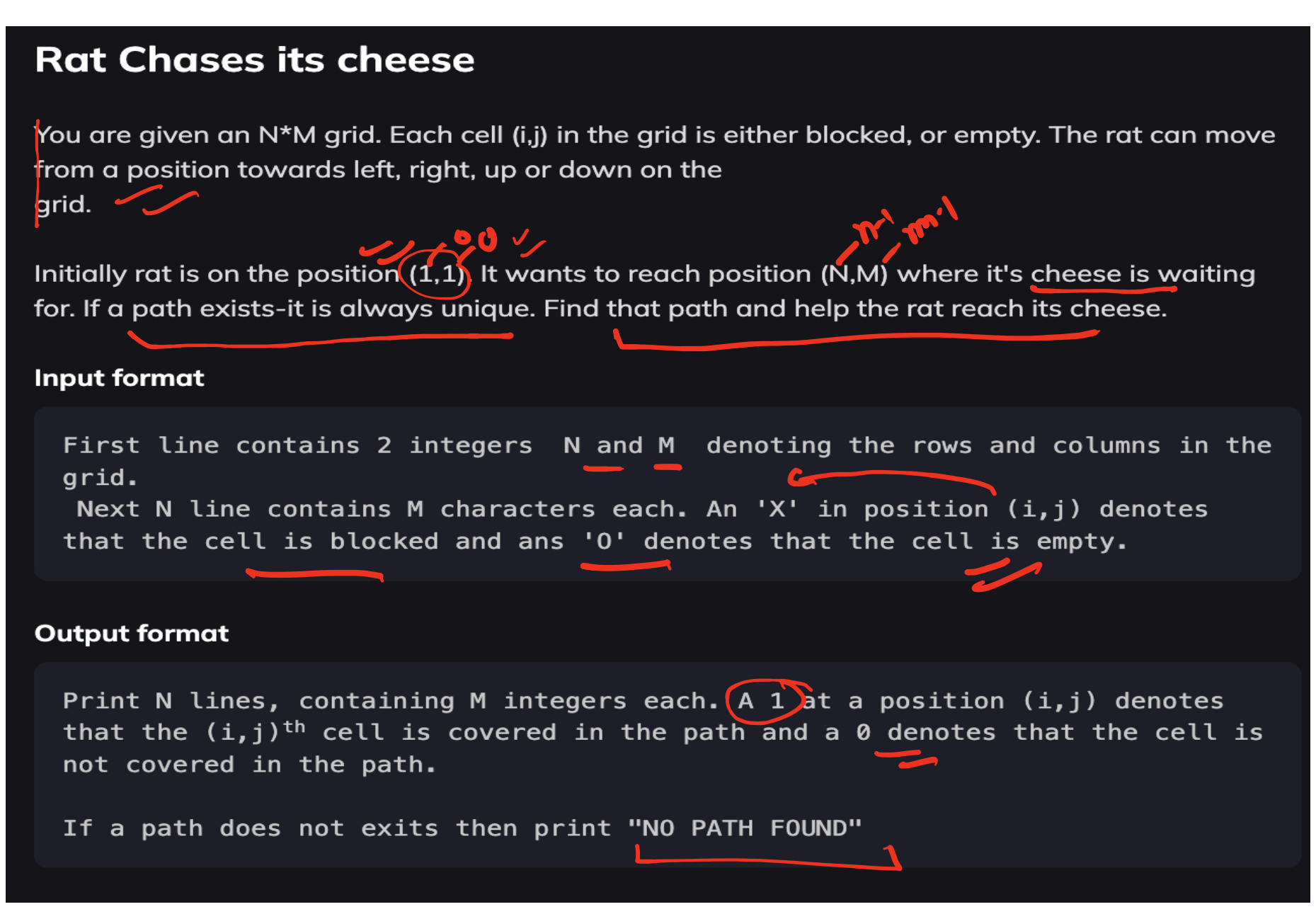
NitiN


NitiN
in N, i
N| itin
Ni tin
nit in
niti N
nitin
N|i|tiN
N|i|t|iN
N|i|t|i|N.
N|i|t|i|n|
1
N|i|t|in|
11
n|it|in
n|it|i|n
n|it|in|
n|it|i|n|
n|i|ti|N
n|i|ti|n|
n|i|tin|
n|iti|n
n|iti|n|
n|itin|
ni| tin
ni|t|in


naan
n| aan
na|an
naa|n
naan
N|a|an
n|aan|
n|a|a|n
n|a|a|n|
n|aa|n
n|aa|n|
i
one
ans
i=1
i=2
i=3
i=4
aan -> (1)
an (2)
a (3)
(4)
ans+N -> (0,1)
ans+Na -> (0,2)
ans+naa (0,3)
(0,4)
naan
-> (0,i)

```
n|i|t|i|n|
n|i|t|in|
n|i|ti|n|
n|i|tin|
n|it|i|n|
n|it|in|
n|iti|n|
n|itin|
ni|t|i|n|
ni|t|in|
ni|ti|n|
ni|tin|
nit|i|n|
nit|in|
niti|n|
nitin|
```

ans+S



Rat Chases its cheese

You are given an N*M grid. Each cell (i,j) in the grid is either blocked, or empty. The rat can move from a position towards left, right, up or down on the grid.

Initially rat is on the position (1,1). It wants to reach position (N,M) where it's cheese is waiting for. If a path exists-it is always unique. Find that path and help the rat reach its cheese.

Input format

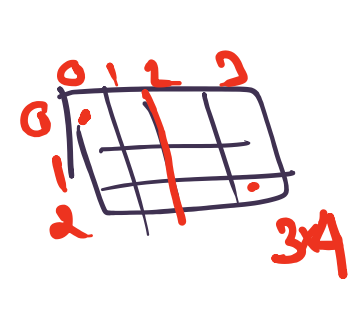
```
First line contains 2 integers  N and M  denoting the rows and columns in the grid.
 Next N line contains M characters each. An 'X' in position (i,j) denotes that the cell is blocked and ans '0' denotes that the cell is empty.
```

Output format

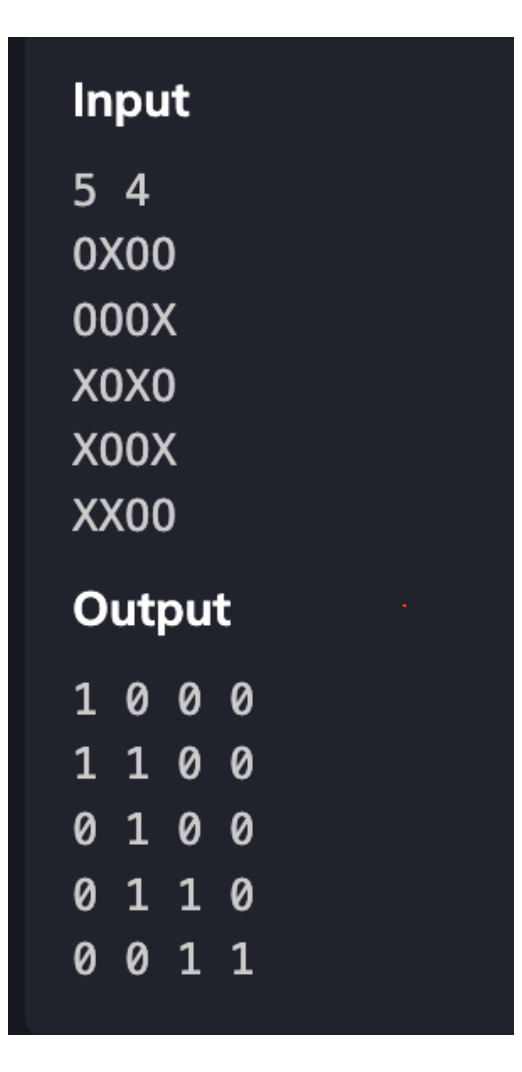
```
Print N lines, containing M integers each. A 1 at a position (i,j) denotes that the (i,j)th cell is covered in the path and a 0 denotes that the cell is not covered in the path.

If a path does not exits then print "NO PATH FOUND"
```

0,0 n-1 m-1



cr<0
cc<0
cr>=
cc>=L

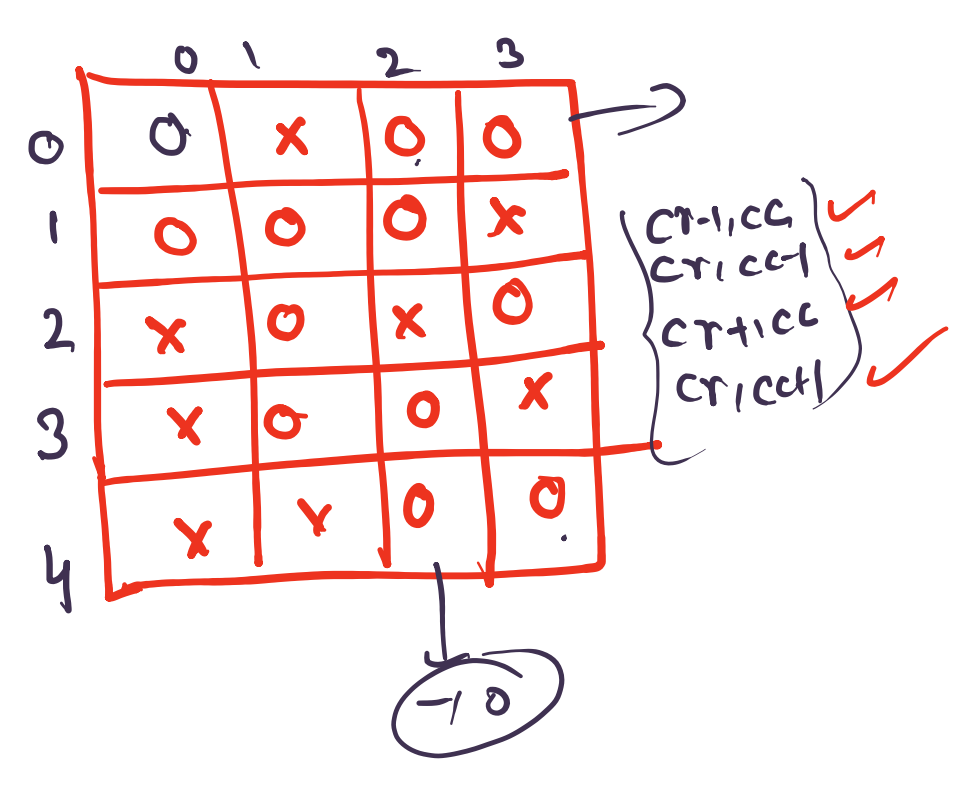


Input

```
5 4
0X00
000X
X0X0
X00X
XX00
```

Output

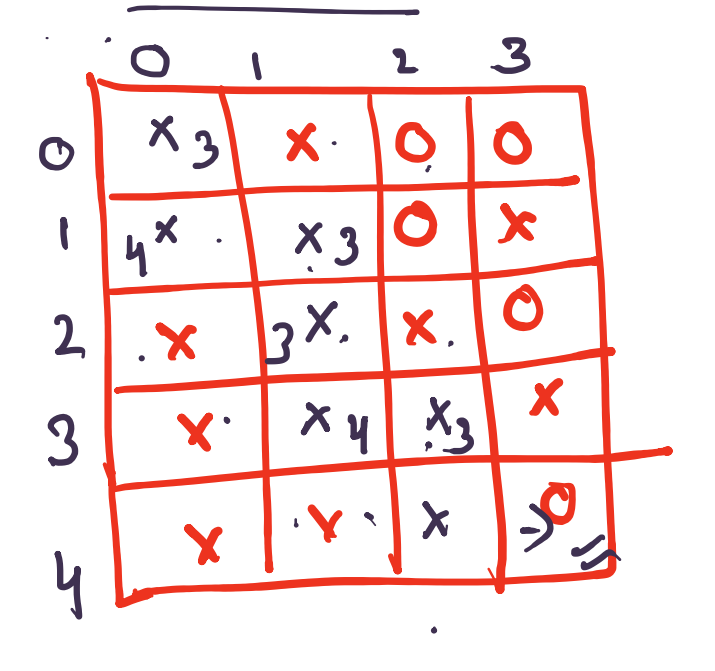
```
1 0 0 0
1 1 0 0
0 1 0 0
0 1 1 0
0 0 1 1
```


0 1 2 3
0 1 2 3 4
cr-1,cc
cr,cc-1
cr+1,cc
cr,cc+1
-1 0

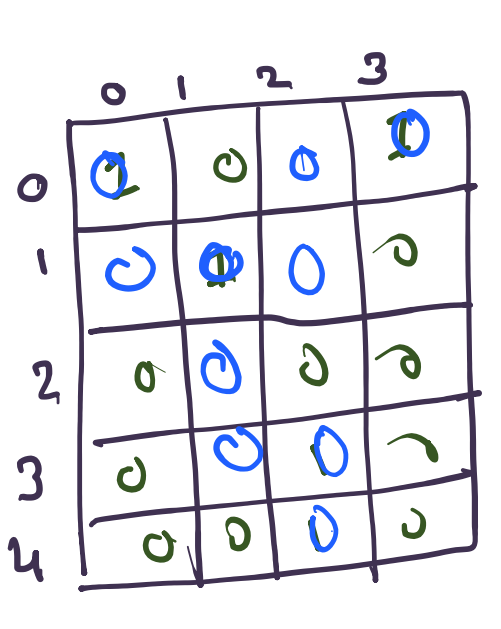
```java
public static void print(char[][] maze, int cr, int cc) {
		// TODO Auto-generated method stub
		if (cr < 0 || cc < 0 || cr >= maze.length || cc >= maze[0].length || maze[cr][cc]=='X') {
			return;
		}
		maze[cr][cc]='X';
		print(maze, cr - 1, cc);// up
		print(maze, cr, cc - 1);// left
		print(maze, cr + 1, cc);// down
		print(maze, cr, cc + 1);// right
		maze[cr][cc]='O';
	}
```

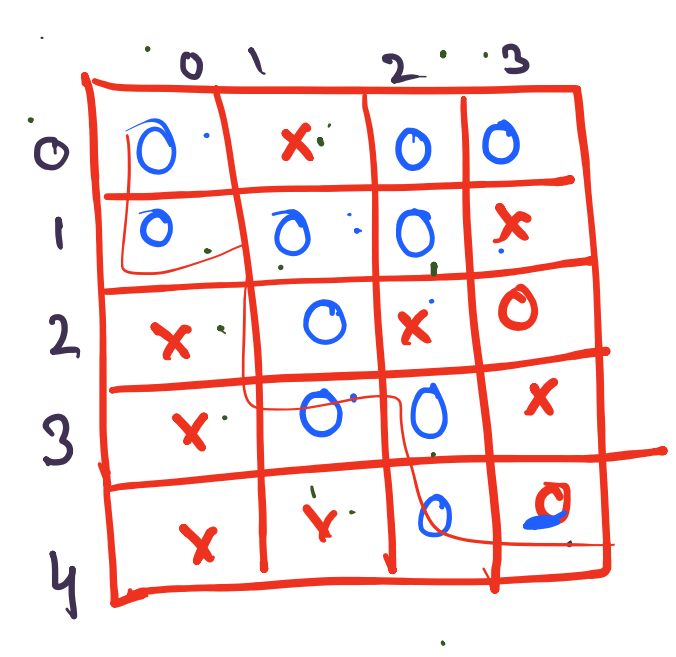
ans[cr][cc]=1

ans[cr][cc]=0



```java
public static void print(char[][] maze, int cr, int cc, int[][] ans) {
		// TODO Auto-generated method stub
		if (cr < 0 || cc < 0 || cr >= maze.length || cc >= maze[0].length || maze[cr][cc] == 'X') {
			return;
		}
		if (cr == maze.length - 1 && cc == maze[0].length - 1) {
			Display(ans);
			return;
		}
		maze[cr][cc] = 'X';
		ans[cr][cc] = 1;
		print(maze, cr - 1, cc, ans);// up
		print(maze, cr, cc - 1, ans);// left
		print(maze, cr + 1, cc, ans);// down
		print(maze, cr, cc + 1, ans);// right
		maze[cr][cc] = 'O';
		ans[cr][cc] = 0;
	}
```

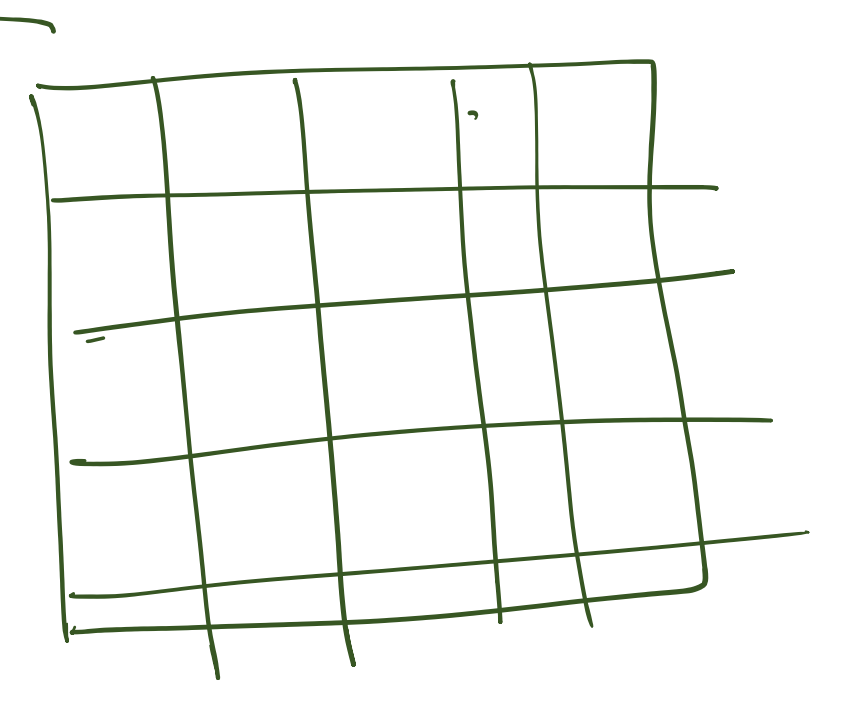




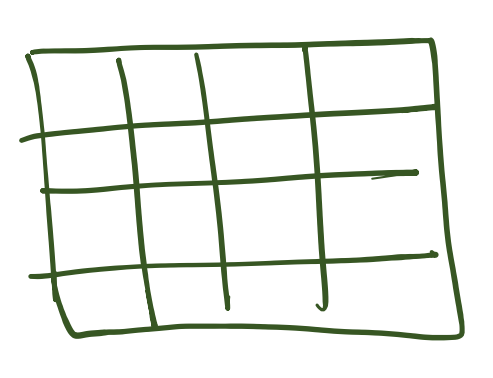
```java
public static void print(char[][] maze, int cr, int cc, int[][] ans) {
		// TODO Auto-generated method stub
		if (cr < 0 || cc < 0 || cr >= maze.length || cc >= maze[0].length || maze[cr][cc] == 'X') {
			return;
		}
		if (cr == maze.length - 1 && cc == maze[0].length - 1) {
			ans[cr][cc] = 1;
			val = 1;
			Display(ans);
			return;
		}
		maze[cr][cc] = 'X';
		ans[cr][cc] = 1;
		print(maze, cr - 1, cc, ans);// up
		print(maze, cr, cc - 1, ans);// left
		print(maze, cr + 1, cc, ans);// down
		print(maze, cr, cc + 1, ans);// right
		maze[cr][cc] = 'O';
		ans[cr][cc] = 0;
	}
```

R = {-1,0,1,0}

C = {0,-1,0,1}

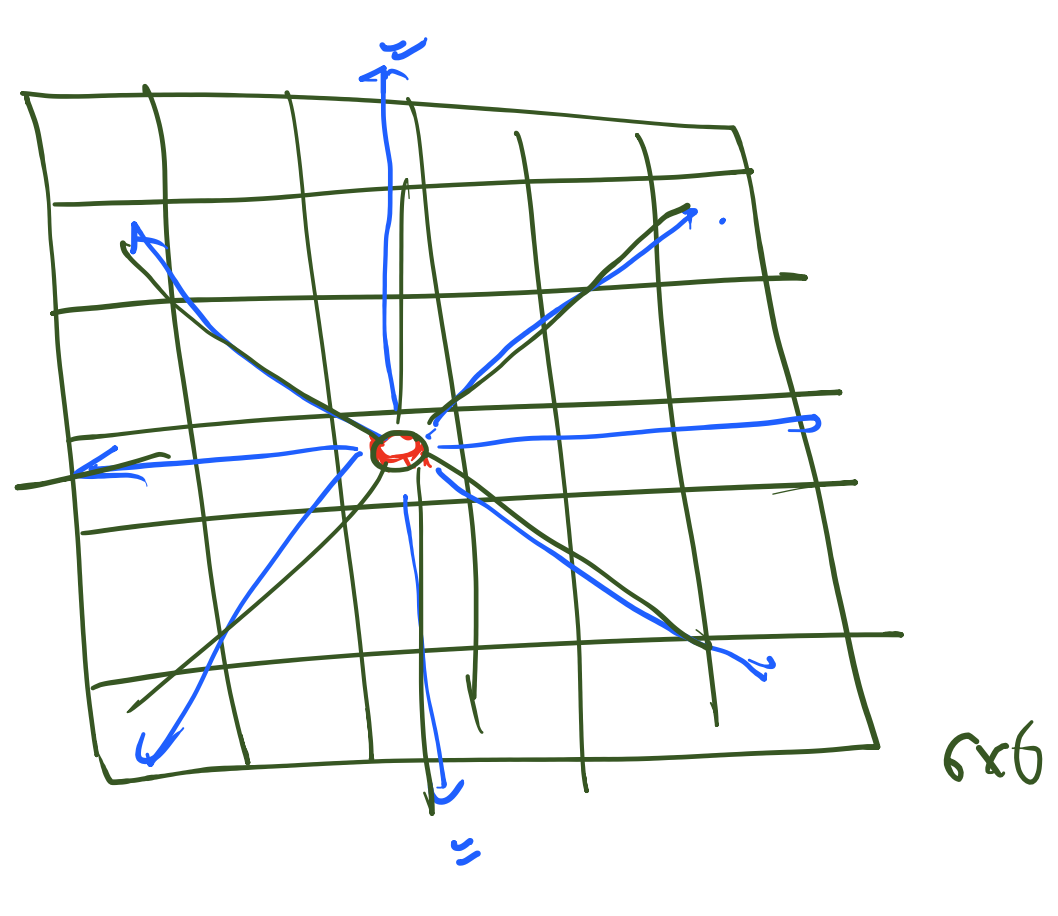


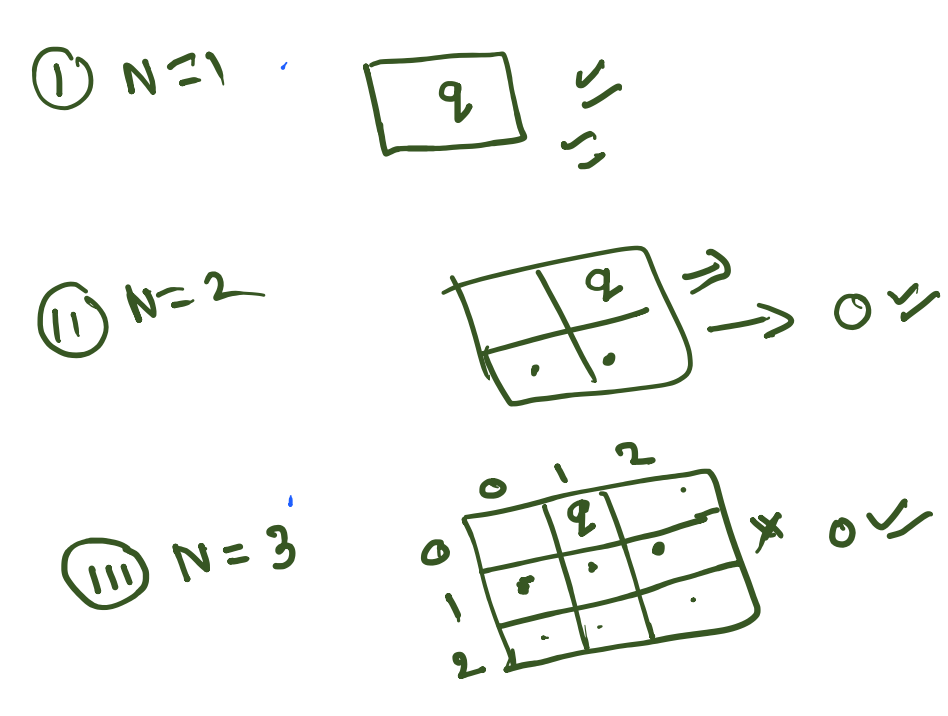
N Queen



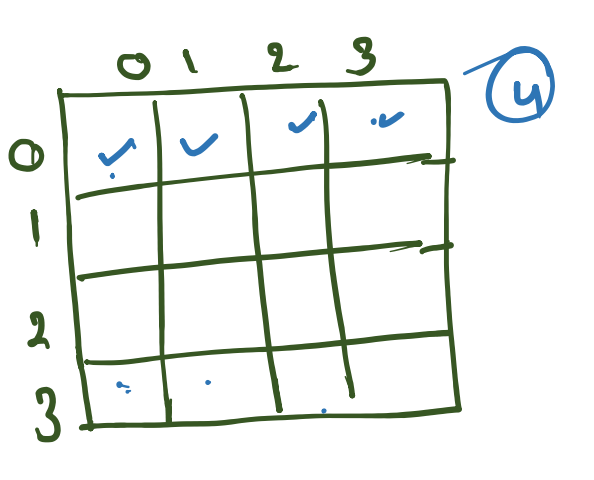
Kill

placed




i N=1
q
ii N=2
q
0
iii N=3
0 1 2
0 1 2
q
0

Row+1



q {
F T F F
F F F T
T F F F
F F T F

q {
F F T F
T F F F
F F F T
F T F F

```java
public static void print(boolean[][] board, int tq, int row) {
		if(tq==0) {
			Display(board);
			return;
		}
		
		for (int col = 0; col < board[0].length; col++) {
			if(isitsafe(board,row,col)) {
				board[row][col]=true;
				print(board, tq-1, row+1);
				board[row][col]=false;
			}
		}
	}
```

F T F F
F F F T
T F F F
F F T F

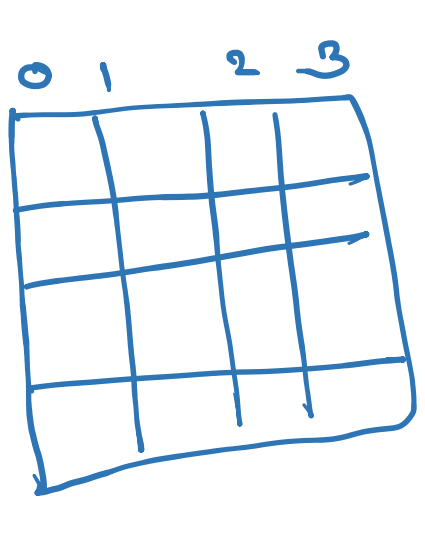
F F T F
T F F F
F F F T
F T F F

1,3

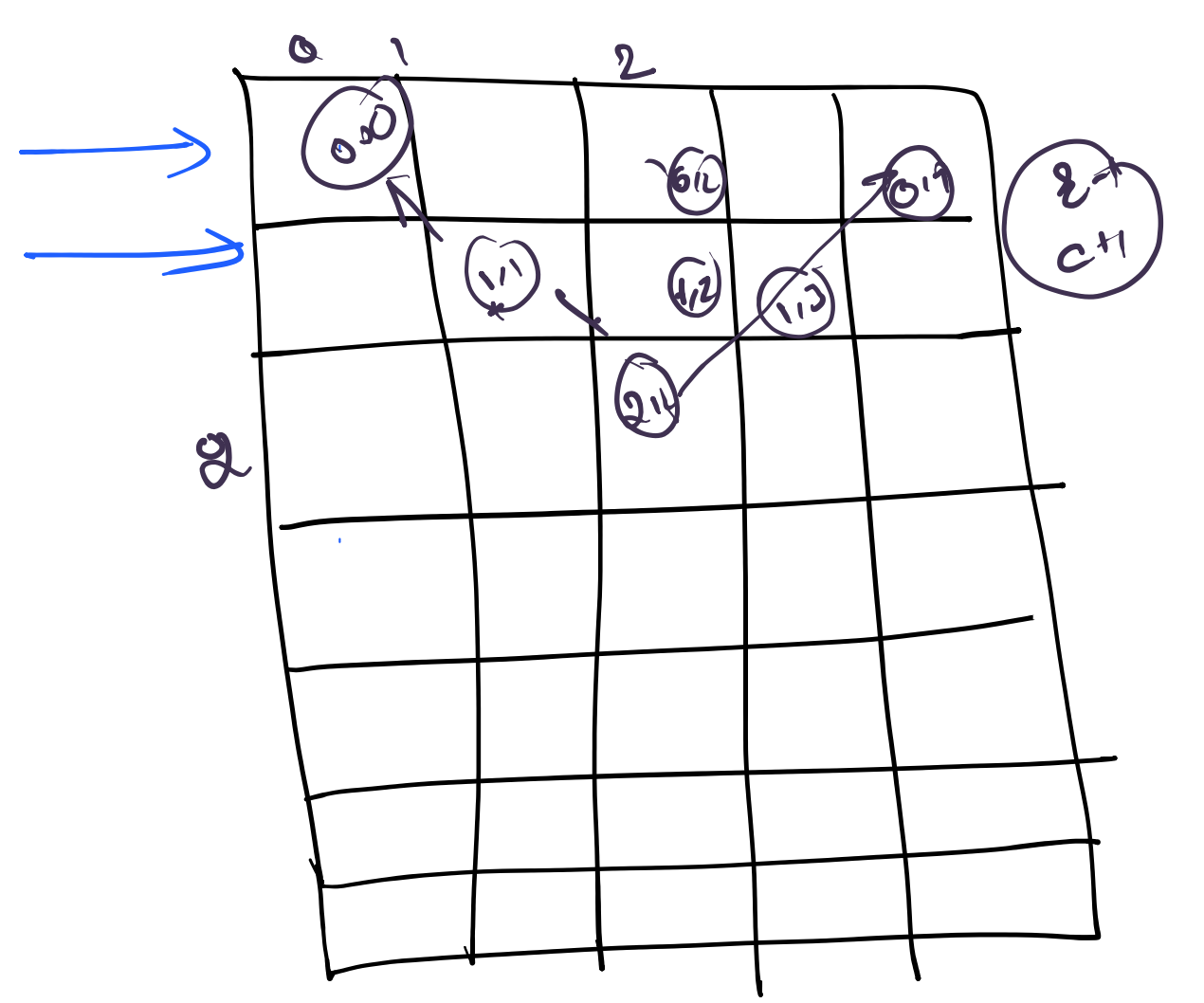
2,2

3,1

4,0



r-c


0 1 2
0,0
0,2
0,3
1,1
1,2
1,3
2,1
2
r+1
c+1